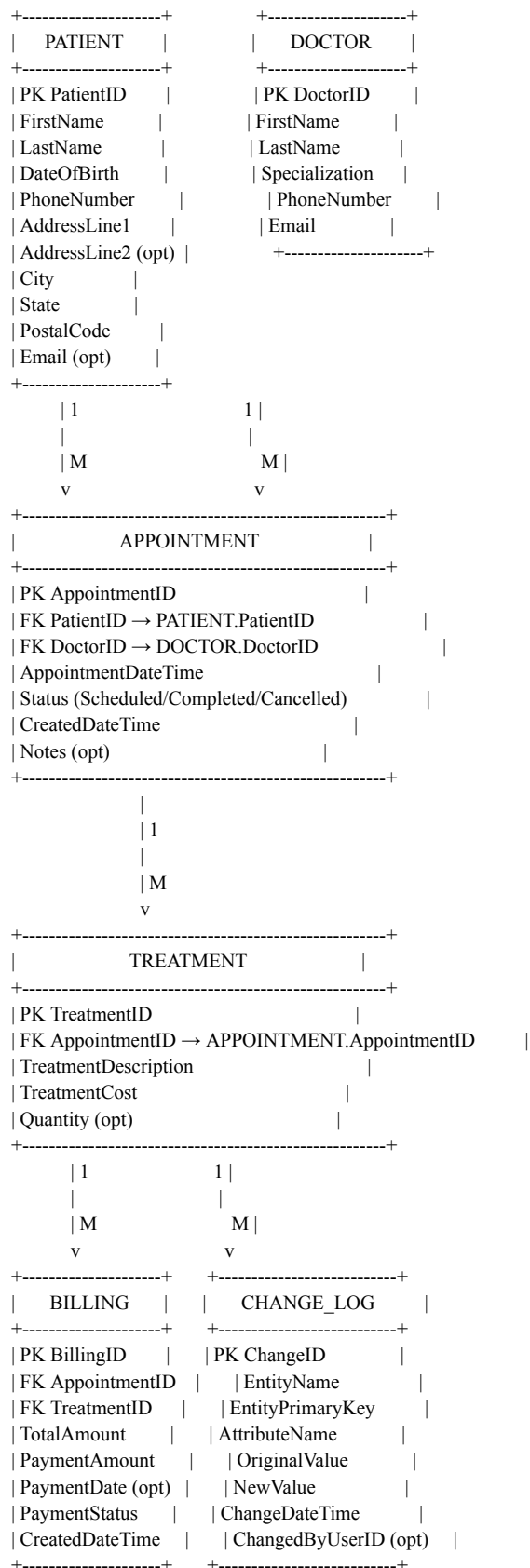


ERD Diagram:



Normalization:

Normalization explanation (1NF, 2NF, 3NF)

First Normal Form (1NF):

Atomic attributes:

PhoneNumber, AddressLine1, City, Specialization, Status, etc. are single values, not lists.

- For example, treatments are separate rows in TREATMENT, not a comma-separated list in APPOINTMENT.

No repeating groups:

- Multiple treatments are stored as multiple rows in TREATMENT (1:M from APPOINTMENT), not as repeated columns (Treatment1, Treatment2, etc.).

Consistent data types:

- Dates, amounts, and text fields are clearly separated and typed.

Second Normal Form (2NF):

How the design satisfies 2NF:

All primary keys are single-column (e.g., PatientID, DoctorID, AppointmentID, TreatmentID, BillingID, ChangeID).

- That means partial dependency cannot occur because there are no composite keys.
- Example: In APPOINTMENT, attributes like AppointmentDateTime and Status depend only on AppointmentID, not on PatientID or DoctorID separately.
- In TREATMENT, TreatmentDescription and TreatmentCost depend on TreatmentID, not just on AppointmentID.

Third Normal Form (3NF)

How the design satisfies 3NF:

PATIENT:

- PhoneNumber, Address, City, State, PostalCode all depend directly on PatientID. There is no attribute that depends on another non-key attribute (e.g., we're not storing derived fields like "Age" that depend on DateOfBirth).

DOCTOR:

- Specialization depends on DoctorID, not on some other non-key attribute.

APPOINTMENT:

- Status, AppointmentDateTime, Notes depend on AppointmentID only. They don't depend on PatientName or DoctorName.

TREATMENT:

- TreatmentCost and TreatmentDescription depend on TreatmentID, not on BillingID or AppointmentDateTime.

BILLING:

- PaymentStatus, PaymentAmount, PaymentDate depend on BillingID. We're not storing derived totals that depend on other non-key attributes (e.g., "RemainingBalance" could be computed in the application).

CHANGE_LOG:

- OriginalValue and NewValue depend on ChangeID, not on EntityName alone.

Business Rules:

Rule 1: Appointment date/time must be in the future when scheduled

Description:

- When creating an APPOINTMENT with Status = 'Scheduled', AppointmentDateTime must be greater than the current date/time.

Why it's needed:

- Prevents scheduling appointments in the past, which would be logically invalid and confusing for staff and patients.
- Enforcing in application logic:
 - On the appointment creation form, validate AppointmentDateTime > NOW().
 - If invalid, show an error and block saving.
 - Allow past dates only when Status is set to 'Completed' and the appointment is being recorded retrospectively.

Rule 2: Patient date of birth must be a valid past date

Description:

- DateOfBirth in PATIENT must be a valid date strictly earlier than today.

Why it's needed:

- Ensures realistic patient records and avoids impossible dates (future birth dates, invalid formats).
- Enforcing in application logic:
 - Use a date picker restricted to past dates.
 - On submit, check that DateOfBirth < TODAY.
 - Reject invalid or future dates with a clear error message.

Rule 3: Treatment cost must be greater than zero

Description:

- TreatmentCost in TREATMENT must be strictly greater than 0.

Why it's needed:

- Prevents nonsensical or negative costs, which would break billing logic and financial reporting.
- Enforcing in application logic:
 - On treatment entry form, validate $\text{TreatmentCost} > 0$.
 - If $\text{TreatmentCost} \leq 0$, show an error and block saving.
 - Optionally enforce a maximum limit if needed by business rules.

Rule 4: Payment amount cannot exceed the total bill amount

Description:

- In BILLING, PaymentAmount must be less than or equal to TotalAmount.

Why it's needed:

- Prevents overpayments being recorded incorrectly and keeps financial data consistent.
- Enforcing in application logic:
 - When creating or updating a BILLING record, check $0 \leq \text{PaymentAmount} \leq \text{TotalAmount}$.
 - If $\text{PaymentAmount} > \text{TotalAmount}$, show an error and prevent saving.
 - For partial payments, allow $\text{PaymentAmount} < \text{TotalAmount}$ and set $\text{PaymentStatus} = \text{'Partial'}$.

Rule 5: A doctor cannot have overlapping appointments at the same time

Description:

- A DOCTOR must not have two APPOINTMENTS that overlap in time (same or overlapping AppointmentDateTime ranges).

Why it's needed:

- Ensures realistic scheduling and avoids double-booking a doctor.
- Enforcing in application logic:
 - When scheduling an appointment, query existing APPOINTMENTS for that DoctorID within the proposed time slot.
 - If any appointment overlaps, reject the new appointment and show a conflict message.
 - Optionally show available time slots to guide the user.

Rule 6: A cancelled appointment cannot have treatments recorded

Description:

- If APPOINTMENT.Status = 'Cancelled', no TREATMENT rows may be created or updated for that AppointmentID.

Why it's needed:

- Prevents recording medical procedures for appointments that never occurred, preserving clinical and billing accuracy.
- Enforcing in application logic:
 - Before inserting a TREATMENT, check the appointment's Status.
 - If Status = 'Cancelled', block the operation and show an error.
 - Optionally, when changing Status to 'Cancelled', warn if treatments already exist and require explicit confirmation or disallow cancellation.

Rule 7: A billing record cannot be marked as “Paid” unless a payment date is recorded

Description:

- In BILLING, if PaymentStatus = 'Paid', then PaymentDate must be non-null.

Why it's needed:

- Ensures that fully paid bills have a clear payment date for auditing and reporting.
- Enforcing in application logic:
 - On save, if PaymentStatus = 'Paid' and PaymentDate is NULL, show an error.
 - Automatically set PaymentDate = TODAY when the user marks a bill as paid, if business rules allow.